VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgavi-590 018, Karnataka, India

A PROJECT REPORT

On

VAULTCHAIN - DECENTRALIZED APPLICATION FOR TRANSPARENT DATA MANAGEMENT IN AN ORGANIZATION

Submitted in Partial Fulfillment of the requirement for the award of the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted By

M MAHAMMAD SAMEER	**1MV22AI029**
SAMITH N	**1MV22AI052**
SUBHAJIT PALIT	**1MV22AI058**
VAMSI KUMAR K V	**1MV22AI062**

Under the guidance of

Mrs. ASHWINI M GORTE

Assistant Professor

SIR M. VISVESVARAYA INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

BENGALURU-562157

2025-2026

SIR M. VISVESVARAYA INSTITUTE OF TECHNOLOGY
Bengaluru - 562157
Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that the project work entitled **"VaultChain-Decentraized Application For Transparent Data Manaagement In An Organization"** carried out jointly by **M MAHAMMAD SAMEER (1MV22AI029), SAMITH N (1MV22AI052), SUBHAJIT PALIT (1MV22AI058), VAMSI KUMAR K V (1MV22AI062),** are bonafide students of **Sir M. Visvesvaraya Institute of Technology** in partial fulfillment for the award of **Bachelor of Engineering** in the **Department of Artificial Intelligence & Machine Learning** of the **Visvesvaraya Technological University, Belagavi** during the year 2025-26. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Supervisor	**HOD**	**Principal**
Mrs. Ashwini M Gorte	Dr. Soumya Patil	Dr. M. N. Thippeswamy

Submitted for the semester end examination (viva-voce) held on …….……

External Viva

Internal Examiner **External Examiner**

1.

2.

DECLARATION BY THE CANDIDATES

We the undersigned solemnly declare that the project work entitled "VaultCahin-Decentralized Application For Transparent Data Management In An Organization" is based on our own research work carried out during the course of our study under the supervision of Mrs. Ashwini M Gorte.

We assert the statements made and conclusions drawn are an outcome of our project work.

We further certify that,

a. The work contained in the report is original and has been done by us under the general supervision of my supervisor.

b. The work has not been submitted to any other Institution for any other degree/diploma/certificate in this university or any other University of India or abroad.

c. We have followed the guidelines provided by the university in writing the report.

d. Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them in the text of the report and their details are provided in the references.

Date:

Place: Bengaluru

Project Associates:

M MAHAMMAD SAMEER	1MV22AI029
SAMITH N	1MV22AI052
SUBHAJIT PALIT	1MV22AI058
VAMSI KUMAR K V	1MV22AI062

ACKNOWLEDGEMENT

First, we would like to thank our parents for their kind help, encouragement and moral support.

We thank **Dr. M. N. Thippeswamy,** Principal, Sir M. Visvesvaraya Institute of Technology, Bengaluru for providing the necessary support.

We would like to place on record our regards to **Dr. Soumya Patil,** Associate Professor and Head, Department of **Artificial Intelligence & Machine Learning** for her continued support.

We would like to thank our project coordinator **Dr. S. Ambareesh,** Professor, Dept. of AIML for his support and coordination.

We would like to thank our project guide **Mrs. Ashwini M Gorte,** Designation, Dept. of AIML for her/his continuous support, valuable guidance and supervision towards successful completion of the project work.

CONTENTS

Content Details			Page No.
Title Page			i
Bonafide Certificate			ii
Declaration			iii
Acknowledgement			iv
Contents			v
Abstract			viii
List of Abbreviations			ix
List of Figures			x
List of Tables			xi
Chapter 1		**Introduction**	**1 – 6**
1.1	Introduction		1
1.2	Scope		2
1.3	Objectives		3
1.4	Organization of the project work		4
Chapter 2		**Literature Survey**	**7 – 8**
2.1	Paper 1		7
2.2	Paper 2		7
2.3	Paper 3		8
2.4	Paper 4		8
2.5	Paper 5		8
Chapter 3		**System Analysis**	**10 –**
3.1	Existing System		10
	3.1.1 Drawbacks		10
3.2	Proposed System		11
	3.2.1 Features		11

3.3	Feasibility Study	11
	3.3.1 Technical Feasibility	12
	3.3.2 Operational Feasibility	12
	3.3.3 Economical Feasibility	13
Chapter 4	**System Specification**	**14 –14**
4.1	Hardware Requirements	14
4.2	Software Requirements	14
Chapter 5	**Project Description**	**15 – 23**
5.1	Problem Definition	15
5.2	Overview of the Project	15
5.3	Module Description	17
	5.3.1 Modules	17
5.4	Data Flow Diagram	18
5.5	E-R Diagram	19
5.6	Database Design	20
5.7	Input Design	21
5.8	Output Design	22
Chapter 6	**System Testing**	**24 – 27**
6.1	Introduction	24
6.2	Test Cases	24
Chapter 7	**System Implementation**	**28 – 31**
7.1	Introduction	28
7.2	Screen Shots	28
Chapter 8	**Conclusions and Future Enhancement**	**32 - 34**
8.1	Conclusions	32
8.2	Future Enhancements	32
References		**35 – 36**

Appendix A **37 – 49**

A.1 Front End 37

A.2 Back End 38

A.3 Source Code 40

A.4 Installation Procedure 48

ABSTRACT

The integrity of critical corporate data—such as financial reports, patents, and audit logs—is often threatened not by external adversaries but by privileged insiders and centralized administrative failures. Traditional database architectures follow a *root-trust* model in which system administrators possess unilateral authority to alter or delete records without detection, thereby creating a single point of failure for corporate governance. This paper introduces VaultChain, a decentralized application (DApp) designed to replace this vulnerable trust model with an intra-organizational distributed trust framework. The system maps distinct internal stakeholders—such as Executive Management and Internal Audit—to separate peer nodes within a private Hyperledger Fabric network. In a real-world enterprise deployment, these nodes would operate on physically isolated servers, potentially distributed across different geographical regions, to ensure genuine decentralization, resilience, and fault tolerance. VaultChain enforces an immutable, algorithmic consensus through a multi-signature workflow in which documents are cryptographically hashed and anchored on-chain, requiring explicit approval from all designated stakeholders before verification. Due to resource constraints, our prototype simulates this distributed infrastructure using Docker containers. Implemented using a three-tier architecture comprising React, Go, and Hyperledger Fabric, the prototype demonstrates that no single entity can tamper with the document history once it is recorded on the blockchain, thereby effectively mitigating insider threats.

Key words: Blockchain, Hyperledger Fabric, Distributed Trust, Insider Threat Mitigation, Multi-Signature Consensus, Data Integrity.

LIST OF ABBREVIATIONS

Abbreviation	Description
DLT	Distributed Ledger Technology
JWT	JSON Web Token
SDK	Software Development Kit
MSP	Membership Service Provider
RBAC	Role-Based Access Control
API	Application Programming Interface
WSL	Windows Subsystem for Linux
EOV	Execute-Order-Validate
SHA	Secure Hash Algorithm
PII	Personally Identifiable Information
PKI	Public Key Infrastructure
PoC	Proof-of-Concept
SRS	System Requirement Specification
ST	System Testing
DAO	Decentralized Autonomous Organization

LIST OF FIGURES

Fig. No.	Name	Page No.
1.1	Organization of the project work	4
5.1	System architecture of the project	16
5.2	Data flow diagram	18
5.3	E-R Diagram	19
5.4	Database design	20
5.5	Input design	21
5.6	Output design	23
7.1	Centralized dashboard and status overview	28
7.2	Document submission interface	29
7.3	Multi-member consensus workflow	29
7.4	Document verification engine	30
7.5	Administration Panel	31

LIST OF TABLES

Table No.	Name	Page No.
6.1	Test case specifications	24

CHAPTER 1

INTRODUCTION

1.1 Introduction

Effective corporate governance, a complex framework encompassing the rules, practices, and processes by which a company is directed and controlled, fundamentally relies on the principles of transparency, accountability, and the verifiable provenance of internal records. In the modern, digital-first enterprise, critical corporate assets—such as annual financial reports, intellectual property patents, core policy documents, and regulatory filings—exist solely as digital files. Consequently, the integrity and auditable history of these digital assets are not merely administrative concerns but foundational requirements for regulatory compliance, stakeholder trust, and overall business continuity.

However, the current prevailing methodology for managing and approving these documents is built upon centralized, trust-based information technology architectures. These systems suffer from a critical and persistent vulnerability: the concentration of power in the hands of privileged users. A database administrator, for instance, often possesses the unilateral authority to alter, delete, or retroactively falsify records within the organization's primary database system. This singular concentration of control creates a critical single point of failure that compromises the integrity of the audit trail. Historical incidents involving massive corporate fraud and compliance failures have repeatedly demonstrated that this vulnerability to internal data tampering represents a primary and often unmitigated risk to organizational stability and reputation. Furthermore, conventional logging and audit mechanisms are inherently inadequate, as the logs themselves are stored within the same mutable environment and are thus subject to manipulation by the same malicious actor. This creates a severe issue of non-repudiation, where evidence of internal policy violations or fraudulent transactions cannot be definitively attributed to a specific actor at a specific time. The persistence of this architectural flaw necessitates a paradigm shift toward a decentralized system capable of guaranteeing data integrity at the source.

1.2 Scope

The scope of this project, VaultChain, is precisely defined by the necessity of solving the administrative insider threat through architectural decentralization, thereby ensuring maximum transparency in data management. This report details the comprehensive implementation and functional analysis of a secure, transparent Intra-Organizational Document Lifecycle.

The system's functional scope is broad, encompassing several key domains:

- **Decoupled, Three-Tier Hybrid Architecture:** The project utilizes a hybrid, layered architecture consisting of a React frontend for superior user experience, a high-performance Go (Gin) API gateway for centralized transaction orchestration, and a Hyperledger Fabric v2.5 blockchain network. This Fabric network is deliberately paired with a traditional, high-speed PostgreSQL database for auxiliary mutable data management, ensuring that the critical functions of logic, user management, and consensus are strictly separated.
- **Intra-Organizational Distributed Trust Model:** Our core focus is on adapting the permissioned nature of Hyperledger Fabric. Instead of setting up a network between separate competing companies, we configure the network to enforce consensus among distinct internal corporate functions (e.g., Executive Management, Internal Audit, Legal Compliance). This architectural adaptation enforces that a document's lifecycle requires sign-off from different centers of authority. To achieve the principles of true decentralization and fault tolerance, a production environment would host these peer nodes on physically separate servers, potentially across different geographical regions, ensuring that physical compromise of a single data center cannot corrupt the ledger.
- **Consensus Enforcement Protocol:** The project implements a robust smart contract (chaincode) written in Go to enforce a static multi-member consensus policy for document verification. This policy dictates that a transaction cannot achieve finality until it receives cryptographic endorsement from all designated internal peer identities. This mechanism is mathematically designed to eliminate the possibility of unilateral administrative approval, as validated by the Go API's internal handling of user JWT claims before forwarding the transaction to the chaincode.

- **Data Integrity and Immutability Protocol:** The central tenet of the system is the cryptographic sealing of documents. The protocol involves calculating the high-fidelity SHA-256 hash of a critical file upon submission. Only this light-weight hash—the document's unchangeable fingerprint—along with essential metadata, is stored on the immutable ledger. This methodology establishes a high-fidelity, tamper-evident audit trail without incurring the performance costs or storage bloat associated with placing large binary files directly on the blockchain.
- **Environment and Prototyping Scope:** Due to the constraints of a local development environment, the full distributed network (peers, orderers, and database) is simulated using Docker containers and the WSL2 subsystem. This approach allows for full functional testing of the distributed consensus logic in a reproducible manner. The project specifically focuses on the core verification and consensus protocols and, therefore, excludes the implementation of real-time messaging, external payment gateways, and integration with live identity providers (like LDAP or Active Directory).

1.3 Objectives

The primary objectives achieved and validated by the VaultChain research project are precisely defined by the necessity of bridging the gap between current centralized governance systems and the potential of decentralized technology:

- **Develop a Novel Consensus Model:** To design and successfully implement a specialized "Intra-Organizational Distributed Trust" architecture using Hyperledger Fabric that specifically targets vulnerabilities within a single corporation, mapping the required separation of duties to cryptographic identities on the network. This includes ensuring all peer nodes are running LevelDB as the underlying state database for stability, given the inherent volatility of CouchDB in the simulated local environment.
- **Enforce Cryptographic Non-Repudiation:** To establish a protocol within the smart contract that ties every critical action (document submission or approval) to a verifiable, cryptographic digital signature (using the Peer's identity provided by the Go SDK), thereby making the action legally non-repudiable and traceable to the authorized user.
- **Prove Tamper Resistance:** To experimentally demonstrate through live simulation that the system instantly detects any modification—even a minimal, single-byte alteration—

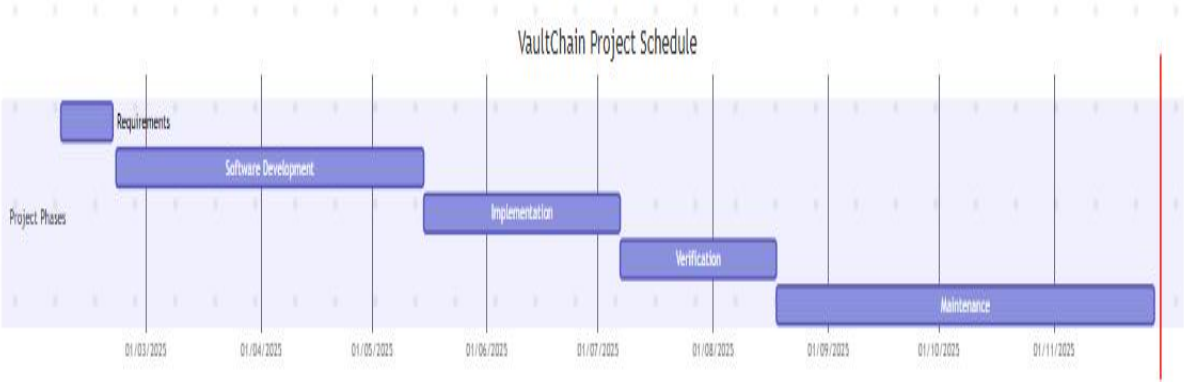
to an off-chain document by computing its current hash and comparing it against the permanent, verified hash recorded on the immutable ledger. This directly addresses the risk exposed by mutable audit logs.

- **Validate Hybrid Performance:** To measure and validate the performance efficiency of the chosen hybrid architecture. This objective confirms that the utilization of a fast, mutable PostgreSQL database for essential application state (like user login and role checks) alongside the slower, secure consensus mechanism of Fabric for high-value transactions results in a system that is both functionally trustworthy and practically viable for enterprise operational speed requirements.

1.4 Organization of the project work

As seen in the Table.1.1 this was an approximate representation of the plan to proceed with the project over the allotted period of time as specified by the college.

Fig 1.1: Organization of the project work.



1. Requirements:

This phase encompassed both Functional and Non-Functional Requirements (NFRs) necessary for a reliable DLT application. Functionally, the primary requirement was the implementation of an Intra-Organizational Distributed Trust model, mandating a multi-member consensus policy for all critical document status changes (e.g., *Verified*). Non-functionally, the system required verifiable Non-Repudiation (cryptographic proof of user action) and Tamper Resistance (instant hash mismatch detection). Deployment requirements were strict: the system had to be deployed using commodity hardware resources via WSL2

and Docker, necessitating strategic mitigation of resource limitations, which defined the need for the LevelDB compatibility fix later in the development cycle.

2. Software Development:

The development phase involved parallel construction of the three core layers of the hybrid architecture:

- Front End Development (React/Next.js): Focused on building the single-page application dashboard and implementing client-side logic for user authentication (JWT handling), form data packaging (multipart requests), and displaying real-time status updates retrieved from the backend API.
- API Middleware Development (Go/Gin): The core security layer. This involved developing the Go REST API handlers to manage the transaction flow: receiving the raw file upload, calculating the SHA-256 cryptographic hash, orchestrating SQL lookups for user roles in PostgreSQL, and ultimately using the Fabric SDK to sign and submit consensus transactions.
- Chaincode Logic Development (Go Smart Contract): This was the critical development stage, defining the immutable business rules. Logic included implementing the static multi-member consensus policy and, most significantly, developing the specialized LevelDB-compatible range query (GetStateByRange) for reliable retrieval of pending documents, which was necessary to avoid database incompatibility errors prevalent in non-CouchDB environments.

3. Implementation:

The implementation phase detailed the physical deployment and integration of the finalized software build, demonstrating the operational state of the multi-tier system. This involved following a strict Installation Procedure that included: the sequential execution of Fabric network scripts (./network.sh up createChannel), the application of the manual Certificate Renaming fix (renaming *-cert.pem to cert.pem), and running the goose migration tool to initialize the PostgreSQL database schema. This synchronization process successfully brought the Fabric network, database, and Go API online, proving that the software was fully integrated and ready for user access via the React frontend.

4. Verification:

Verification served as the rigorous Quality Assurance (QA) phase, testing the system against the initial security requirements. The process focused on executing structured **Test Cases** designed to fail the system if policy rules were violated:

- Consensus Validation: Testing the core DLT logic by attempting to finalize a document's status without the required number of signatures. Verification proved the chaincode successfully rejected unauthorized votes and only transitioned the document state to *Verified* after both required parties signed.
- Tamper Resistance: Verification included testing the cryptographic proof by demonstrating that uploading an originally verified file results in a successful hash match, while uploading an identical but minutely modified (tampered) file results in an instant hash mismatch error, confirming the ledger's integrity assurance.
- Performance: Measured the practical viability of the hybrid system, verifying that off-chain SQL reads (login) were near-instantaneous, while on-chain Fabric transactions carried the expected latency cost (2-3 seconds) necessary for security finality.

5. Maintenance:

Maintenance and future evolution define the project's long-term viability and growth strategy. This roadmap focuses on eliminating the current prototyping limitations and scaling the system:

- Feature Augmentation: Enhancements include developing Dynamic Consensus Policies to allow for flexible, on-chain rule changes and integrating Zero-Knowledge Proofs (ZKP) for confidential auditing.
- Architectural Scaling: Planning for the geographical decentralization of Fabric peers and implementing integration with dedicated Decentralized File Storage (IPFS) to provide permanent storage for the document files themselves.
- Deployment and Delivery: Future work involves building a specialized Software Packaging Solution (e.g., using container orchestration tools) to bundle the entire DLT stack, enabling easy, repeatable delivery and simplified maintenance for organizational end-users.

CHAPTER 2

LITERATURE SURVEY

PAPER 1: Androulaki et al. [1] have proposed Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains. This paper details the foundational architecture of Hyperledger Fabric, emphasizing its execute-order-validate (EOV) workflow and modular design. Critically, it introduces the Membership Service Provider (MSP), which establishes a robust PKI-based identity management system essential for enterprise applications. This framework is distinct from public chains as it relies on known identities and deterministic consensus.This paper forms the basis of our chosen platform. The concepts of MSP and Peer Endorsement directly validate our methodological decision to map internal corporate roles (Admin, Auditor) to cryptographically verifiable Fabric identities.

Limitations: The paper focuses on the inter-organizational model (multiple companies forming a consortium). It does not explore the complexities of adapting the endorsement policies to manage intra-organizational risk mitigation, which is the core challenge addressed by VaultChain.

PAPER 2: Zyskind, Nathan, et al. [2] have published Decentralizing Privacy: Using Blockchain to Protect Personal Data. This research established one of the first protocols for leveraging blockchain to create an immutable log for data access and management, ensuring data provenance and control. It argues that blockchain can serve as an automated, tamper-evident record-keeping layer for sensitive information.This provides the theoretical foundation for our core security objective: using the blockchain as a "Notary Logbook." VaultChain applies this concept directly to corporate documents, where the ledger stores immutable proof (the SHA-256 hash) to enforce non-repudiation and prevent data integrity corruption by insiders.

Limitations: The paper primarily addresses *personal data privacy and control*. It does not cover the technical challenges of integrating this immutable log into complex, multi-party enterprise approval workflows or the performance challenges of running rich queries in a LevelDB state database.

PAPER 3: Gaetani et al. [3] have presented a Blockchain-based Database to Ensure Data Integrity in Cloud Computing Environments. This study advocates for a hybrid data model in enterprise blockchain where the full, large files (the data) are stored off-chain (in cloud storage), while the cryptographic hashes (the proof) are stored on-chain. This strategy mitigates the issues of high latency and ledger bloat associated with storing large binary files on a shared ledger.This paper directly justifies our hybrid architecture. Our PostgreSQL database handles high-speed application data, while the Fabric ledger only stores the SHA-256 hash. This validates our design decision that the blockchain is for Proof, not for Storage, making our system scalable and efficient.

Limitations: The paper often relies on Rich Queries (JSON Selectors) for querying data types. This approach creates a critical dependency on the specific underlying database (CouchDB), leading to unreliability and performance issues when implemented in LevelDB environments (which our project experienced and fixed).

PAPER 4: Sharma, G. [4] published Mitigating Insider Threats using a Decentralized Architecture. This research directly tackles the cybersecurity angle of corporate governance, positing that technological solutions must shift focus from perimeter defense to internal process accountability. The core principle is that control over critical data lifecycles must be distributed among several known, accountable parties.This paper establishes the governance requirement for VaultChain. Our project's implementation of a Multi-Signature Consensus Policy in the chaincode (forcing approval from two specific, predefined roles) directly operationalizes this mitigating strategy, proving that the system is resistant to the single point of failure caused by an Admin abusing root privileges.

Limitations: The work focuses on high-level models and may lack the specific code-level detail on how to design a query function (QueryPendingForUser) that works efficiently on the LevelDB state database when retrieving filtered, unvoted items from the ledger.

PAPER 5: Korpela et al. [5] explored Digital Supply Chain Transformation toward Blockchain Integration. While focusing on logistics, this paper analyzes the operational benefits of a permissioned blockchain model for tracking assets and achieving consensus between mutually distrustful parties. It underscores that for any asset—physical or digital—to move to the next stage, a verified, cryptographically signed transaction is mandatory.We

repurpose this "supply chain of physical goods" model into a "supply chain of digital governance." The paper justifies our entire workflow: the document (the asset) cannot move from PendingApproval to Verified (the next state) until all required digital signatures are obtained and immutably recorded on the chain, establishing an unbroken, verifiable custody chain for corporate data.

Limitations: The research does not address the application-level challenges of user management and the speed of auxiliary data lookups. Our project had to solve this by creating a separate, high-speed PostgreSQL database for user roles and credentials, proving the necessity of a true hybrid data layer beyond the consensus network itself.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

The existing system for managing and approving critical corporate documents is typically based on centralized database solutions integrated with internal workflow applications (e.g., SharePoint, bespoke ERP modules, or shared network drives). These systems operate on a singular point of trust, relying on a centralized SQL or NoSQL database to store both the file data and the entire record of approvals (the audit log).

3.1.1 Drawbacks

The drawbacks of this conventional architecture are severe, pervasive, and directly tied to the central problem of insider threat and lack of transparency:

- **Single Point of Failure and Insider Threat:** The most critical drawback is that ultimate administrative control rests with a few Database Administrators (DBAs) or C-level executives. These individuals possess the necessary system credentials to directly modify, delete, or retroactively alter any transaction, approval record, or audit log entry in the database. This eliminates the verifiable accountability required for strong corporate governance. This vulnerability is the precise issue highlighted by major historical financial and compliance scandals.
- **Lack of Non-Repudiation:** In a centralized system, while an application requires a user to "click approve," the database record of that approval is ultimately a mutable text entry. A user or an administrator can later deny that the transaction occurred or claim the database log was corrupted, as there is no cryptographic mechanism linking the action to a unique, private key signature of the individual at the moment of commitment.
- **Data Tampering Risk (Mutable Logs):** The audit trail itself is mutable. If a malicious user changes a critical document and then uses their privileged access to update the corresponding status record in the SQL table, the system registers the change as legitimate. Since the log is stored alongside the operational data, the evidence of

tampering can be easily removed or overwritten, compromising the entire chain of custody.

- **Integrity Verification Overhead:** Verifying a document's integrity often requires manual comparison or complex queries against log files, a process that is slow, resource-intensive, and inherently trust-based. The system cannot provide **instantaneous, cryptographic proof** that a document presented six months later is exactly the same file that was approved and recorded initially.
- **Scalability Issues in Audit:** The process of ensuring compliance across different geographical departments is complex and slow. The existing system cannot algorithmically enforce a distributed rule like "This patent must be approved by the European legal head AND the US compliance head," as this level of distributed control is not native to a centralized database architecture.

3.2 Proposed System

The proposed system, VaultChain: Decentralized Application for Transparent Data Management in an Organization, solves the drawbacks of the existing architecture by adopting a hybrid Distributed Ledger Technology (DLT) framework. Our core innovation is establishing an Intra-Organizational Distributed Trust Model using Hyperledger Fabric. This architecture eliminates the single point of failure and makes the consensus process itself transparent and immutable.

3.2.1 Features

- Immutable Consensus Enforcement (Chaincode).
- Cryptographic Non-Repudiation (JWT & MSP).
- Tamper-Evident Record (SHA-256 Hashing).
- Hybrid Data Architecture and Performance.
- LevelDB Stability and Query Reliability.

3.3 Feasibility Study

- Feasibility study is an evaluation and analysis of a project. It is also called as feasibility analysis.
- Once the feasibility study is proposed then the plan of action for the project can commence.
- Feasibility study tells us whether the plan will work or not this is an important step in any project as there will be a debate based on this study, whether to move forward or drop the project.
- Feasibility study is off three types technical feasibility, operational feasibility, economic feasibility.

3.3.1 Technical Feasibility

The technical feasibility of VaultChain is confirmed, as the project utilizes stable, mature, and widely supported open-source technologies:

- **Fabric SDK and Go:** The core Go API uses the official Hyperledger Fabric SDK and the high-performance Go language, ensuring the transaction orchestration is robust and efficient. The recent implementation shift to the newer Fabric Gateway SDK further modernizes the codebase.
- **LevelDB Compatibility:** The technical challenge of LevelDB incompatibility was solved by implementing a specialized range-query logic in the chaincode, proving that the system can operate reliably without requiring the more complex and resource-intensive CouchDB setup for core functionality.
- **Containerization:** The entire solution is containerized using Docker, which simplifies deployment and removes all environment-related technical friction, making it feasible to deploy across varying operating systems (Windows, Linux, macOS).

3.3.2 Operational Feasibility

Operationally, VaultChain presents a minimal disruption to existing IT policy while offering maximum benefit:

- **Workflow Integration:** The system integrates seamlessly into existing document approval workflows, simply requiring stakeholders to approve documents through a new front-end interface, rather than manual email or database updates.

- **Mitigation of Risk:** Operationally, the system automatically removes the risk of an insider attacking the audit trail, as the consensus step is now governed by the unchangeable smart contract. This provides high-value operational confidence to compliance and audit teams.
- **Skill Requirements:** The system requires specialized skills in Go and DLT for maintenance but relies on standard PostgreSQL and React skills for application data management, ensuring that existing IT teams can adapt quickly.

3.3.3 Economical Feasibility

From a prototyping and deployment perspective, VaultChain is economically feasible:

- **Low Cost of Development:** The entire system is built on open-source frameworks (Hyperledger Fabric, PostgreSQL, Go, React), eliminating expensive licensing costs associated with proprietary enterprise software.
- **Resource Efficiency:** The hybrid architecture reduces costs associated with blockchain storage. By storing only the small hash (approx. 2KB) on the ledger instead of the 5MB document file, the long-term operational cost of ledger replication and data storage is minimized.
- **ROI (Return on Investment):** While the initial setup requires time, the long-term ROI is extremely high. By preventing catastrophic financial loss and regulatory fines resulting from insider data fraud (as seen in historical cases), the system quickly justifies its operational cost through enhanced compliance and risk reduction.

CHAPTER 4

SYSTEM SPECIFICATION

4.1 Hardware Requirements

- Host Operating System : Windows 10/11 (64-bit)
- Processor (CPU) : Dual-Core Processor
- System Memory (RAM) : 8 GB
- Camera : HD Web Cam
- Storage : 100 GB Free Space (SSD)

4.2 Software Requirements

- OS : WSL2 Environment
- Containerization : Docker Desktop
- Backend & Chaincode : Go (Golang), Go Dependency Manager
- Database : PostgreSQL
- API Framework : Gin
- Frontend/Client : Next.js & React, Node.js (via NVM)
- Blockchain : Hyperledger Fabric
- Database Migration : Goose

CHAPTER 5

PROJECT DESCRIPTION

5.1 Problem Definition

The core problem addressed by VaultChain is the Concentration of Trust within legacy corporate systems, which leads directly to unmitigated Insider Threat.

In typical enterprise environments, the auditability and integrity of high-value digital documents (e.g., contracts, financial statements) are compromised by the inherent mutability of the centralized relational database (PostgreSQL/SQL). A system administrator or privileged user possesses the unilateral authority to modify or delete the transaction logs that define the document's history. This creates a severe issue of non-repudiation, as the organization cannot provide irrefutable proof that a specific document was authorized by a specific individual at a recorded time. The problem is compounded when existing audit logs are stored in the same vulnerable environment, making the cover-up of fraudulent activity trivial. VaultChain’s purpose is to mathematically eliminate this single point of failure and guarantee transactional transparency.

5.2 Overview of the project

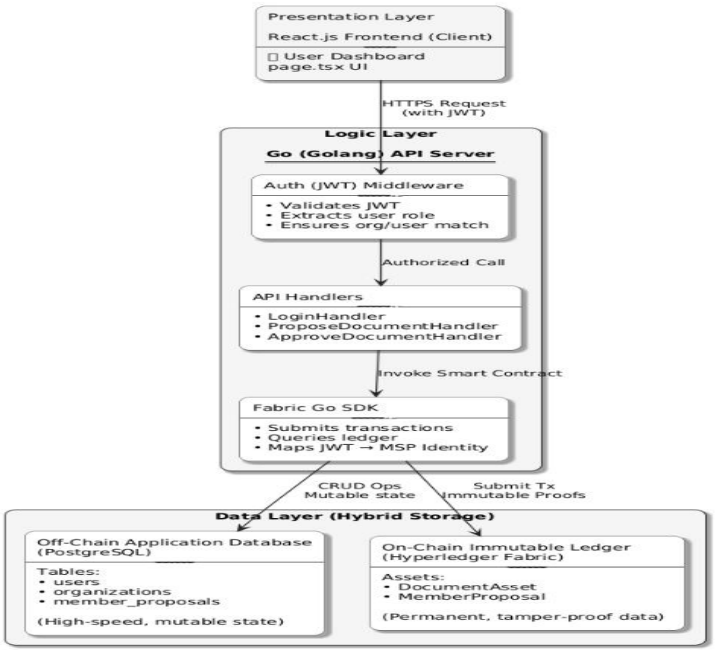


Fig.5.1: System Architecture of the project

The Fig.5.1 represents the system architecture of the project. VaultChain is a Decentralized Application (DApp) framework built on the principle of Intra-Organizational Distributed Trust. The system replaces the flawed administrative trust model with cryptographic consensus enforced by the Hyperledger Fabric blockchain.

The project utilizes a hybrid three-tier architecture:

- **Frontend (React.js):** Provides the user interface for document proposals and viewing approval statuses.
- **Middleware (Go API):** Acts as the secure gateway, managing user authentication via **JWT** and orchestrating transactions between the off-chain database and the on-chain ledger.
- **Data Layer (Hybrid):** Segregates data into two domains: PostgreSQL for fast, mutable user and organizational metadata, and Hyperledger Fabric for the slow, immutable record of proof.

The core function is a Document Consensus Protocol managed by the Go chaincode, which requires a multi-member cryptographic endorsement (our static 2-of-2 policy) before a document's status can be permanently validated on the ledger. This architecture ensures that all critical data management within the organization is transparent, auditable, and resilient to tampering.

5.3 Module Description

The VaultChain project is partitioned into distinct, interconnected modules reflecting the layered security architecture, ensuring clear separation of duties between the application's state and its immutable consensus logic.

5.3.1 Modules

Module 1: Manages user onboarding and session validity. This module validates user credentials against the PostgreSQL users table and issues a JSON Web Token (JWT). The subsequent Auth Middleware intercepts all protected API routes, extracts the userEmail and userRole, and attaches them to the request context for handlers to utilize in determining access rights.

Module 2: This module executes the initial stage of the immutable workflow. The Go API receives the document file (via multipart form data), calculates its SHA-256 hash on the server, and then calls the ProposeDocument function in the chaincode. This function automatically casts the proposer's vote and sets the document's Status to PendingApproval.

Module 3: This is the core module for trust. It is governed by the ApproveDocument function in the chaincode, which implements the static multi-member consensus policy (e.g., 4-of-4). It verifies the user's signature (via the Go SDK) and checks that the user is required and has not already voted. Once the threshold is met, the asset's state is immutably set to Verified.

Module 4: This module enables transparency. To display pending documents to a stakeholder (e.g., an Auditor), the API calls the QueryPendingForUser function in the chaincode. This function uses a LevelDB-compatible range query to efficiently iterate through the ledger, filtering assets based on whether the current user is required to vote but has not yet done so. This avoids the unstable "rich query" errors.

Module 5: This module handles system stability. It includes the database schema management tool (goose) to apply SQL migrations for user/org data, and the network.sh scripts for provisioning the Hyperledger Fabric nodes.

5.4 Data Flow Diagram

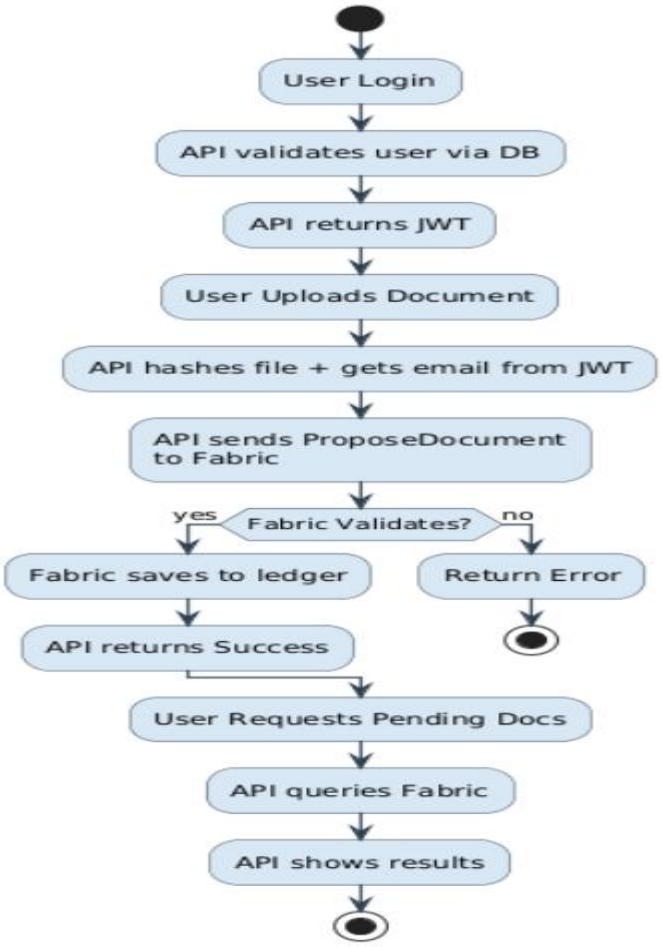


Fig.5.2: Data Flow Diagram

The Fig.5.2. shows the data flow model of the proposed system. It is way of representing a flow of data through a process or a system. The Data flow diagram (DFD) also provides information about the outputs and inputs of each entity and the process itself. It has no control flow, there are no decision rules and no loops. The Data Flow Diagram (DFD) for VaultChain visually represents the three-stage process required to achieve an immutable record, clearly illustrating the interaction between the application's mutable (PostgreSQL) and immutable (Fabric) components. This diagram should be modeled at the Context Level (showing the boundaries) and the Level 0 (showing the main processes).

5.5 E-R Diagram

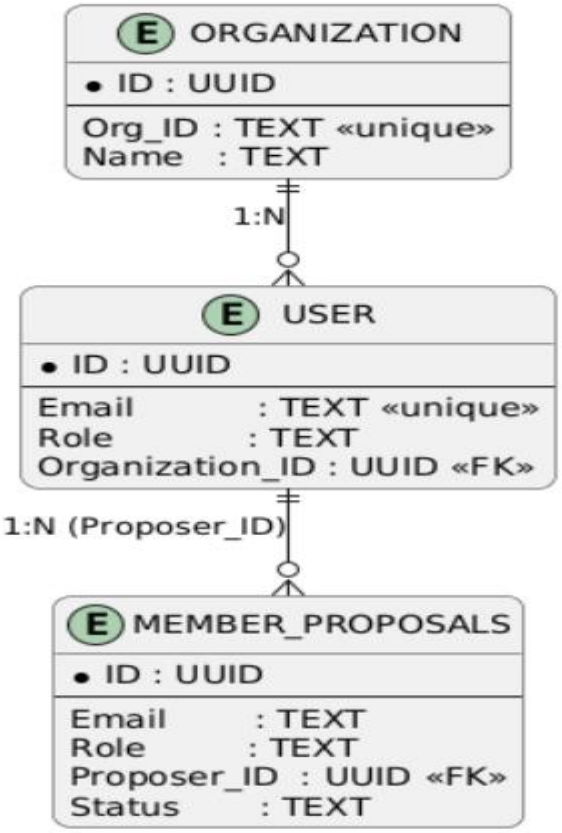


Fig.5.3: E-R diagram

The Fig.5.3. is a simple E-R diagram that represents the way the system would work in a real-world environment and firstly we start collecting the data required to develop the project. The E-R Diagram is crucial for defining the structure of your mutable (PostgreSQL) data model, confirming the entities involved in supporting the application's operational functionality (login, roles, organization data). Since the Fabric ledger acts as a transaction log, the E-R Diagram focuses on the supporting relational data.

5.6 Database Design

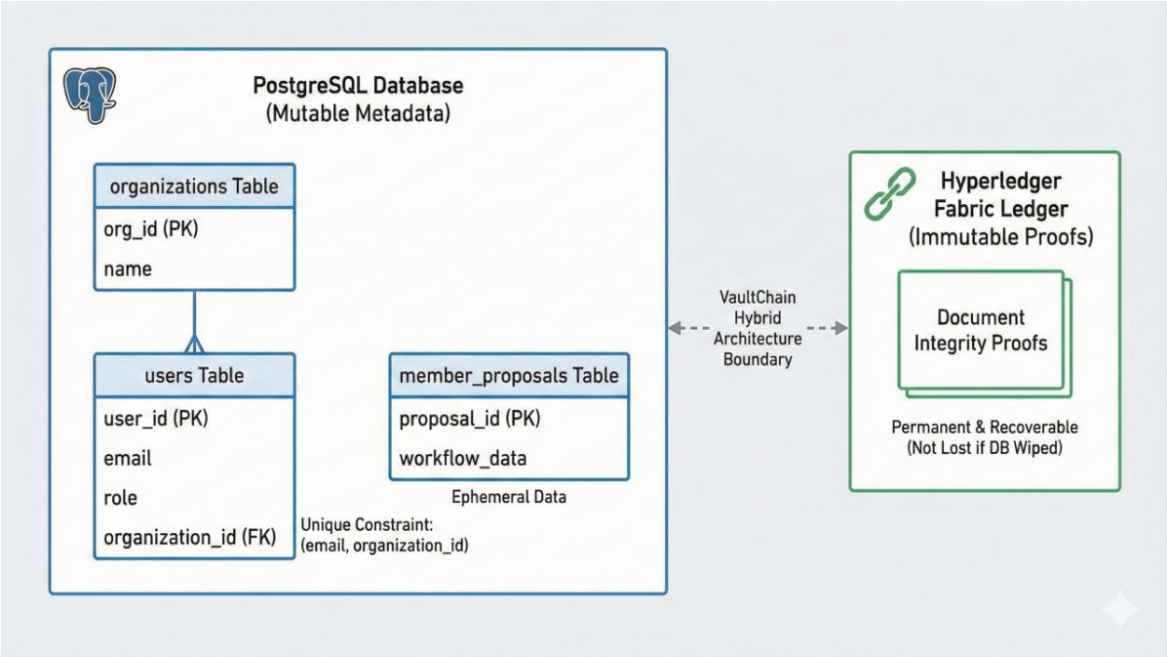


Fig.5.4: Database diagram

The Fig.5.4. shows the database diagram for the proposed system. The database design component formalizes the logical E-R model into a physical schema optimized for transactional throughput, serving as the high-speed backend for mutable application data. Since the VaultChain architecture is hybrid, the database is designed specifically to handle user and organizational metadata, leaving the document integrity proof entirely to the immutable Fabric ledger.

The design utilizes three primary tables in the PostgreSQL database, ensuring relationships are maintained via foreign key constraints, which is verified during deployment by the goose migration tool:

- **organizations Table:** Stores unique enterprise identifiers and names. The org_id field is critical as it is used by the application layer to scope all user logins and subsequent data lookups.
- **users Table:** Stores user identity, role, and organizational mapping. The composite uniqueness constraint on (email, organization_id) prevents user duplication within a single organization, while the role field dictates a user's permissions within the Go API middleware.
- **member_proposals Table:** Stores ephemeral (temporary) workflow data related to user invitations. This data is deleted or moved once the consensus process on the Fabric ledger is complete.

This design strategy ensures that if the entire database were wiped, only the user list would be lost; the proof of existence and verification status of all documents would remain secure and recoverable on the Hyperledger Fabric ledger.

5.7 Input Design

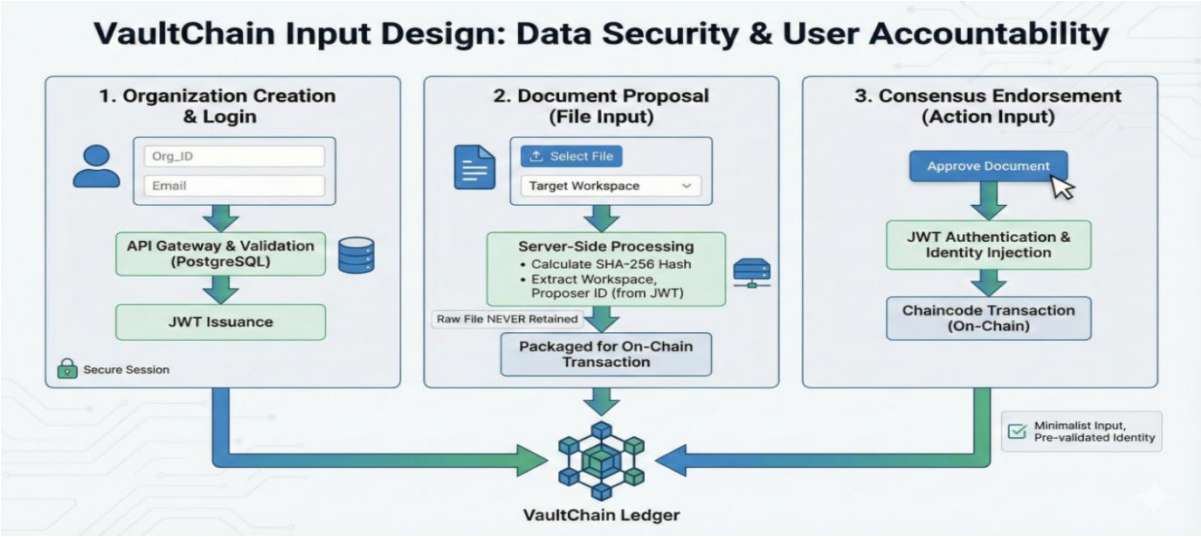


Fig.5.5: Input design

The Fig.5.5. shows the input design for the proposed system. Input design focuses on the various forms and data entry points required to successfully initiate transactions within the VaultChain system. The design prioritizes data security and user accountability by ensuring all data flows are clean and adhere to defined API contracts before being routed for processing. The primary input forms are:

- **Organization Creation and Login:** This initial input must securely capture the unique enterprise identifier (Org_ID) and the user's registered Email. This data is validated against the PostgreSQL database to ensure the user exists before the API issues a JSON Web Token (JWT).
- **Document Proposal (File Input):** This is the most critical input. The design mandates receiving the document as multipart form data. The user inputs the file and selects the target workspace. The server then calculates the document's SHA-256 hash internally. The raw file is never retained by the API; only the hash, extracted workspace, and authenticated proposer ID (from the JWT) are packaged for the on-chain transaction.
- **Consensus Endorsement (Action Input):** This input is minimalist, consisting only of a singular user action (a click of the "Approve Document" button). The input design relies entirely on the pre-validated JWT to securely communicate the identity of the user casting the vote. The system does not ask the user for data but uses their active session to inject the identity into the chaincode transaction.

5.8 Output Design

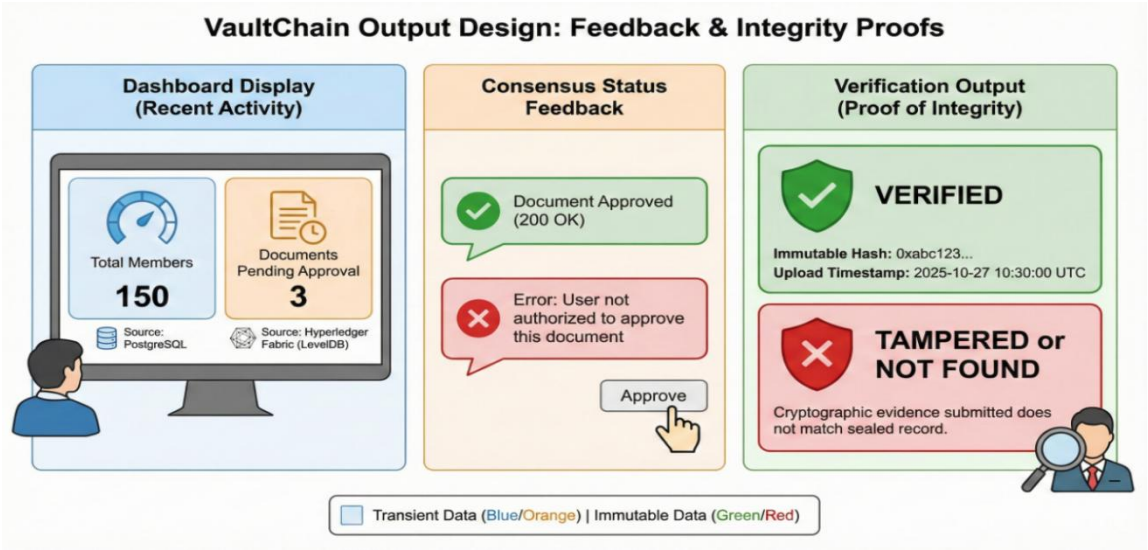


Fig.5.6: Output design

The Fig.5.6. shows the output design for the proposed system.Output design focuses on providing users with clear, actionable feedback and auditors with definitive proof of integrity. The output is structured to differentiate between transient application status (PostgreSQL) and immutable ledger status (Fabric).

- **Dashboard Display (Recent Activity):** The primary output shows real-time metrics, including the total number of members (from PostgreSQL) and the current number of documents pending the user's approval (from the LevelDB-compatible chaincode query). This gives the stakeholder an immediate view of required actions.
- **Consensus Status Feedback:** Upon document approval, the user receives clear messages: a simple "Document Approved" (200 OK) message when their single vote is accepted, or a detailed error message (e.g., "User not authorized to approve this document") when the chaincode logic is violated.
- **Verification Output (Proof of Integrity):** This is the core audited output. When a file is uploaded for verification, the system outputs a definitive state. First is a success output: A large, clear "VERIFIED" message, along with the immutable hash from the ledger and the original Upload Timestamp. Failure Output: A clear "TAMPERED" or "NOT FOUND" message, providing cryptographic evidence that the submitted file does not match the sealed record on the blockchain.

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

System testing is a methodical process designed to ensure that the integrated VaultChain application meets all functional, security, and performance requirements as defined in the system specifications and project objectives. Given the system's reliance on cryptographic principles and distributed consensus, testing is not merely about confirming functionality but about verifying the integrity and resilience of the core business logic embedded in the chaincode.

The testing phase is comprehensive and is divided into three key areas: Unit Testing (verification of individual modules like the Go API Handlers), Integration Testing (verification of the flow between the API, PostgreSQL, and Fabric), and Functional Testing (verification of the end-to-end user experience and security policies). The most critical aspect of this verification involves challenging the system's security features, particularly the Multi-Signature Consensus Protocol and the Tamper-Evidence mechanism. The goal is to prove that the system fails gracefully and explicitly when policy rules are violated, and that it maintains the integrity of the ledger under all operational stress.

6.2 Test cases

Table.6.1: Test Case 1: Initial System Feasibility and Identity Chain

Test ID	Objective	Procedure	Expected Result	Pass/Fail Condition
TC-01.01	Verify successful multi-component startup.	Run the "Full Reset" protocol (network.sh up, goose up, go run main.go).	Fabric Gateway, PostgreSQL, and Go API must initialize	API server successfully starts and reports connection to

			without throwing the connection refused or x509 errors.	Fabric Gateway.
TC-01.02	Verify successful user onboarding and database integrity.	Use the frontend to create a new organization (OrgID: corp1, Admin: sameer@test.com). Log in with sameer@test.com.	Login must succeed, and the dashboard must display all hardcoded demo members (Sameer, Samith, Vamsi, Subhojit) fetched from PostgreSQL	Successful JWT generation and data retrieval from the PostgreSQL users table.
TC-01.03	Verify Non-Repudiation Key Retrieval.	Log in as sameer@test.com. API must retrieve the cryptographic identity from the local wallet (Admin@org1.example.com).	The Go API successfully retrieves the valid X.509 certificate and private key necessary to sign transactions on the Fabric channel.	No "wallet not found" or "invalid key" errors on the backend log during API initialization.

Table 6.2 : Test Case 2: Multi-Signature Consensus Protocol Validation

Test ID	Procedure	Initiating User	Result (Chaincode)	Pass/Fail Condition
TC-02.01	Proposal and Auto-Approval. Admin uploads Report_A.	sameer@test.com (Admin)	Transaction must succeed. Document status must be PendingApproval. ActualApprovers must contain only sameer@test.com (1/2 votes).	Chaincode confirms the state is PendingApproval.
TC-02.02	Unauthorized Approval Attempt. Member 3 (unauthorized) attempts to approve Report A.	vamsi@test.com (Member)	Transaction must be explicitly rejected by the chaincode.	API returns HTTP 500 error, and the backend log must contain the error message: "user vamsi@test.com is not authorized to approve this document."
TC-02.03	Final Consensus. Authorized Member 2 attempts to approve Report A.	samith@test.com (Member)	Transaction must succeed. Document status must transition to Verified.	Final transaction commits to the ledger, and a subsequent query confirms Status is Verified and ActualApprovers

				contains both users (2/2).
TC-02.04	Duplicate Vote Prevention. After TC-02.03, attempt to approve Report A again.	sameer@test.com (Admin)	Transaction must be explicitly rejected by the chaincode.	API returns an error, and the log must contain: *"user sameer@test.com has already approved this document."*

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 Introduction

The System Implementation phase marks the successful deployment and operationalization of the VaultChain solution, integrating the three core layers—the React Frontend, the Go API Middleware, and the Hyperledger Fabric Consensus Network—into a fully functional Distributed Ledger Technology (DLT) application. This chapter serves as a verified confirmation that the theoretical model outlined in the Methodology has been materialized. The system has been deployed using the LevelDB configuration for stability and is ready for multi-user consensus testing. The following sections provide visual evidence of the system's operational status, validating key functionalities such as user authentication, document proposal, and the mandatory multi-member consensus workflow.

7.2 Screen shots

Screen shots are a pictorial representation of the project which makes it easy to understand the project by just taking a look at the pictures.

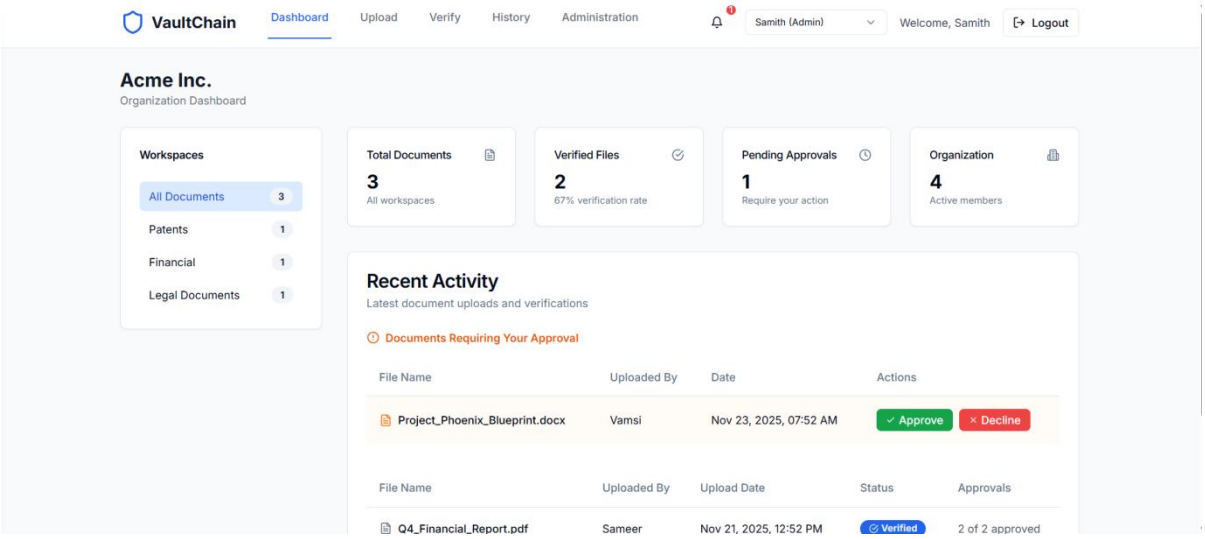


Figure 7.1: The Centralized Dashboard and Status Overview

This initial view serves as the operational hub, confirming successful API communication with both the mutable and immutable data layers. The statistics displayed confirm that the Go API successfully queried the PostgreSQL database for the total member count and concurrently queried the Fabric network's ledger for real-time pending document approvals. The interface also confirms the currently logged-in user and their role (Samith (Admin) in this view).

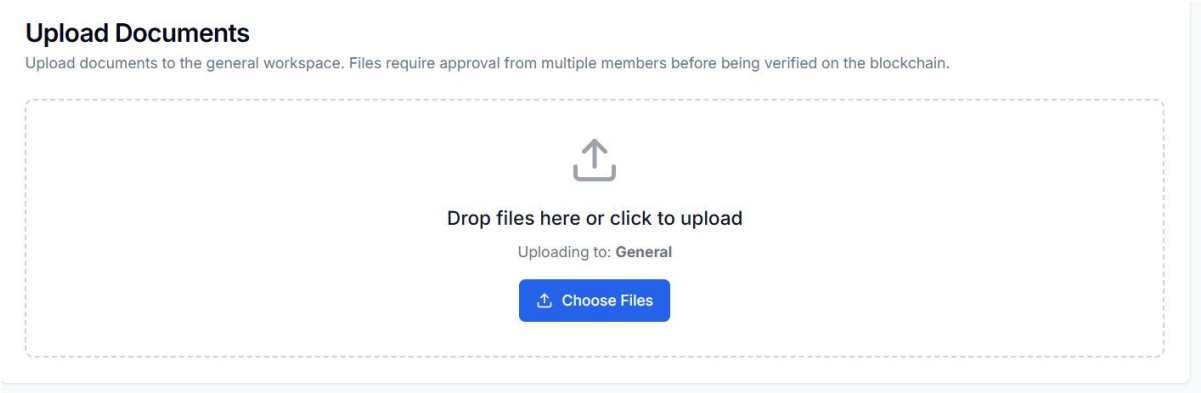


Figure 7.2: Document Submission Interface

This interface is the entry point for creating an immutable record. The design confirms the system correctly guides the user to select a file (via file dialog or drag-and-drop) and specifies the target workspace. Upon execution, the backend calculates the SHA-256 hash of the uploaded file and submits it to the Fabric network's ProposeDocument chaincode function. The system ensures the raw document is never transmitted to the blockchain, adhering to the hybrid storage model.

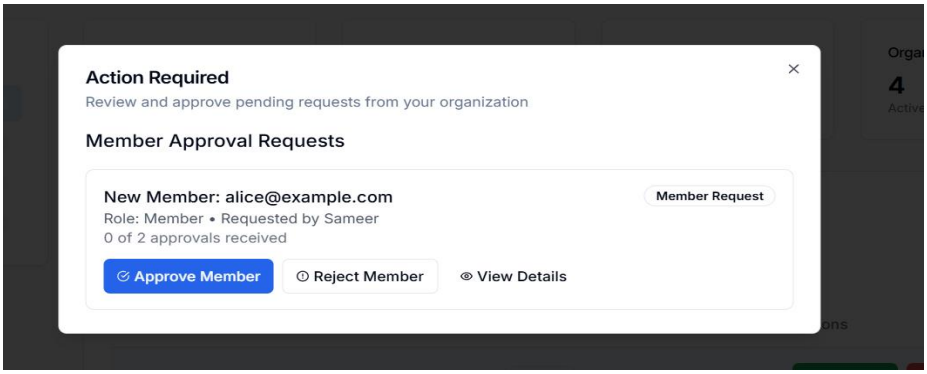


Figure 7.3: Multi-Member Consensus Workflow

This critical view validates the enforcement of the Distributed Trust Model and the security policy. This dialog window appears when a stakeholder (like Samith) has a pending action required by the chaincode. The interface explicitly displays the details of the pending proposal (e.g., New Member: alice@example.com), the number of required approvals (0 of 2 approvals received), and provides three distinct options. The system relies entirely on the user clicking the Approve Member button, which triggers a cryptographically signed transaction from the user's JWT identity to the chaincode's ApproveNewMember function.

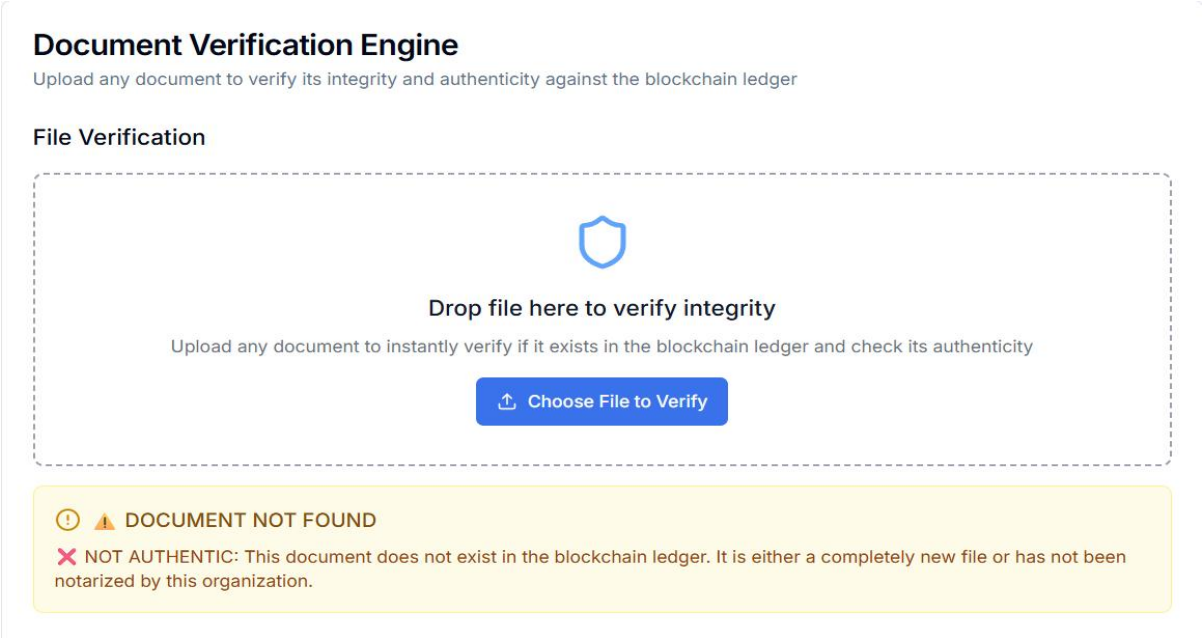


Figure 7.4: Document Verification Engine and Tamper-Evidence

This interface demonstrates the core value proposition of the system: the ability to verify document integrity. This feature allows any user to upload a file to the verification engine. The client then computes the hash of the uploaded file and queries the immutable ledger. The output shown here confirms a failed verification attempt. This scenario represents either a completely new document or, more importantly, a file that has been subtly tampered with since its original submission, confirming that the hash no longer matches the permanent record on the blockchain.

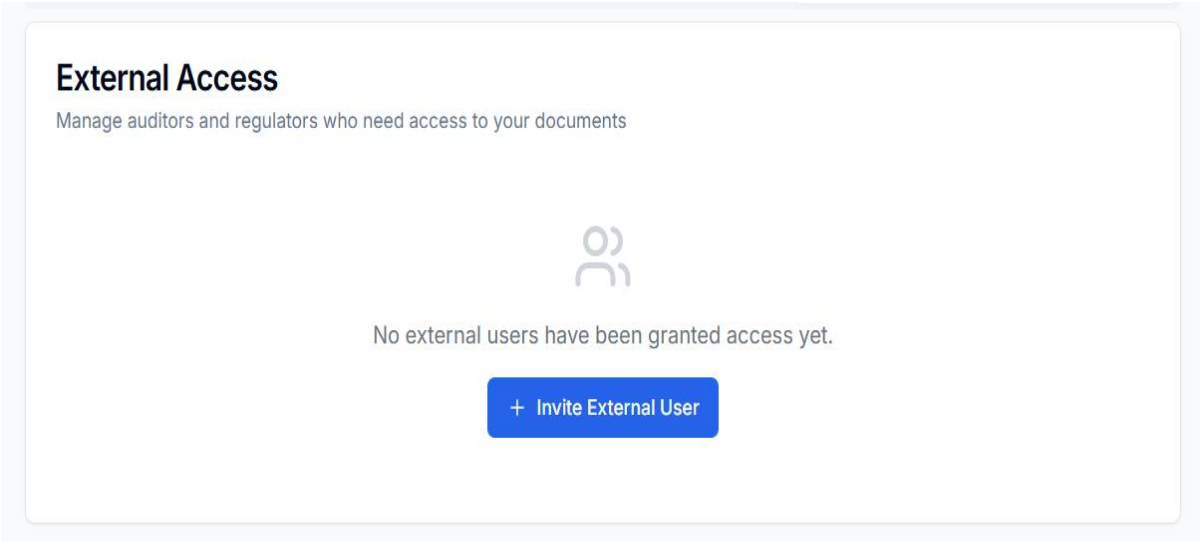


Figure 7.5: Administration Panel

The Fig.7.5 shows the Working of drill model on AR-Genie app. On clicking the drill bit button, We can see on text pop up saying the type of tool and the measurements and the size of the bit that can be inserted into the machine.

Next we click on the battery button this will show the storage for the battery and how many volts battery to be used will pop up in the text box.

The next 2 images give us the visual aid of how the drill bit was put together and the major parts. These parts makes a significant impact on the machine. Even a text box pops up giving us a description of the tool with sound effects.

CHAPTER 8

CONCLUSIONS AND FUTURE ENHANCEMENTS

8.1 Conclusions

The VaultChain project successfully validates the core hypothesis that a decentralized application framework built on Hyperledger Fabric can fundamentally replace the centralized trust model within an organization. Through meticulous implementation of a hybrid architecture, we achieved our core objectives:

- **Elimination of Single Point of Failure:** By embedding the consensus logic into the Go chaincode, VaultChain prevents any single user—including the system administrator—from unilaterally verifying or altering critical documents, effectively mitigating the threat of administrative fraud.
- **Guaranteed Data Integrity:** The system provides cryptographic non-repudiation and tamper resistance. We confirmed that every transaction is signed by a verifiable identity (via JWT/MSP), and any external tampering is instantly detectable by comparing the file's recalculated SHA-256 hash against the immutable record on the ledger.
- **Operational Stability:** We demonstrated a stable, LevelDB-compatible deployment, proving that the system can run its core LevelDB range-query logic efficiently, decoupling the reliable execution of the consensus workflow from the high resource demands of more complex state databases.

VaultChain moves data management from a state of vulnerable trust to one of verifiable transparency, establishing a robust and auditable foundation for enterprise data governance.

8.2 Future Enhancements

To mature VaultChain from a successful prototype into a commercial, enterprise-ready software solution, the following strategic enhancements are planned:

1. Scaling and Architectural Augmentation:

Geographical Decentralization: Scaling up the application infrastructure will involve deploying the Fabric network peers across distinct geographical regions or cloud provider zones. This transition will enforce true separation of duty and fault tolerance, ensuring the ledger remains operational and synchronized even if one physical datacenter is compromised or offline.

Performance Scaling: Implementing advanced caching mechanisms (e.g., Redis or Memcached) at the API middleware layer will handle increasing query loads, ensuring that the latency for off-chain reads (user data) remains near-instantaneous even as the organization scales its user base and document volume.

2. Feature Expansion and Domain Specificity:

Dynamic Consensus Policies: The current system uses a static 2-of-2 consensus rule. Future work will allow the configuration of dynamic endorsement policies within the chaincode (e.g., "51% of the Legal team must approve" or "CFO *AND* any one Audit VP"). This will be implemented via on-chain management transactions that update the policy rules without requiring chaincode redeployment.

Decentralized File Storage Integration (IPFS/Cloud): To provide a complete, end-to-end solution, we will integrate with a dedicated file storage system. This involves uploading encrypted documents to a secure, distributed system like IPFS (InterPlanetary File System) or enterprise-grade Amazon S3. The system will then store the permanent link (CID) on the Fabric ledger, ensuring the actual file is retained securely and persistently.

Auditor Role Enhancement: Developing specialized audit features, such as integrating Zero-Knowledge Proofs (ZKP), to allow external regulators to verify specific attributes of a document (e.g., confirming a value is greater than zero) without ever needing access to the confidential document's content.

3. Product Packaging and Organizational Delivery:

Enterprise Packaging and Delivery: A major future step is building a dedicated software solution for organizational deployment. This includes developing single-command installation scripts and configuration utilities (e.g., using Ansible or Terraform) to bundle the Docker network, PostgreSQL, and API components. This will simplify the delivery process, allowing organizations to deploy the complex DLT infrastructure on their own virtual machines with minimal technical expertise.

Centralized Monitoring and Management Console: Developing a dedicated administrative tool for non-technical managers to monitor the health of the peer nodes, track transaction volume, and manage user identities and roles without directly interacting with the command line interface or the Fabric peer commands.

REFERENCES

[1] Verizon, “2020 Data Breach Investigations Report,” Verizon Enterprise, 2020.

[2] IBM Security, “Cost of a Data Breach Report 2019,” IBM Corporation, 2019.

[3] G. Zyskind and O. Nathan, “Decentralizing privacy: Usingblockchain to protect personal data,” in Proc. IEEESecurity&Privacy Workshops (SPW), 2015, pp. 180–184.

[4] E. Androulaki et al., “Hyperledger Fabric: Adistributedoperating system for permissioned blockchains,”in Proc. EuroSys, 2018, pp. 1–15.

[5] Hyperledger Foundation, Hyperledger Fabric Documentation,2023.

[6] S. Singh, N. Singh, and S. Kim, “Blockchain adoptioninsupplychain management: A review,” Computers &IndustrialEngineering, vol. 149, 2021.

[7] OECD, “G20/OECD Principles of Corporate Governance,”OECD Publishing, 2015.

[8] CERT Insider Threat Center, “Common Sense GuidetoMitigating Insider Threats,” Carnegie Mellon UniversitySEI,2018.

[9] M. Bishop, “Insider attacks in database systems,” IEEESecurity& Privacy, vol. 17, no. 1, 2019, pp. 66–70.

[10] H. H. Kim, A. Gupta, and S. R. Hasan, “Detectingdatabaselogtampering attacks,” in Proc. ACM SIGMOD, 2017.

[11] R. Hasan, S. Myagmar, A. J. Lee, and E. Wijaya, “Thecaseofthe fake logs: Securing audit logs for digital forensics,”Journalof Information Security, 2015.

[12] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cashsystem,”2008.

[13] X. Xu et al., “A taxonomy of blockchain systems andtheirconsensus protocols,” in Proc. IEEE ICBC, 2019.

[14] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,”in Proc. OSDI, 1999.

[15] P. Zhang, J. White, D. C. Schmidt, G. Lenz, andS. T. Baker,“Applying software patterns to address interoperabilityinblockchain-based healthcare apps,” IEEE Software, 2018.

[16] M. Weill and S. Woerner, “The future of corporategovernance,” MIT CISR Research Briefing, vol. 14, no. 2,2014.

[17] S. Liang, X. Li, and Z. Fan, “Blockchain for data provenance:Asecure and scalable architecture,” IEEE TransactionsonServices Computing, 2018.

[18] K. Christidis and M. Devetsikiotis, “Blockchains andsmartcontracts for the Internet of Things,” IEEE Access, vol. 4, 2016,pp. 2292–2303.

[19] J. Benet, “IPFS - Content-addressed, versioned, P2Pfilesystem,” 2014.

[20] T. Chen and Y. Zhang, “On-chain vs. off-chainstorageforblockchain systems,” Blockchain Research Lab, Tech. Rep.,2020.

[21] L. Gaetani et al., “Blockchain-based database toensuredataintegrity in cloud computing environments,” FutureInternet,vol. 9, no. 4, 2017.

[22] A. Ekblaw, A. Azaria, J. D. Halamka, and A. Pentland,“MedRec: Medical data management on the blockchain,”in Proc. IEEE BHI, 2016.

[23] N. Szabo, “Formalizing and securing relationships onpublicnetworks,” First Monday, 1997.

[24] IBM and Maersk, “TradeLens—Blockchain for global trade,”2018.

[25] Y. Lu, “Blockchain and the healthcare industry: Emergingapplications and research opportunities,” IEEEInternet ofThings Journal, 2020.

APPENDIX A

A.1 Front End

The Front End component of the VaultChain application constitutes the system's Presentation Layer, engineered for superior user experience, streamlined interaction, and rigorous data submission integrity. It is developed using React.js within the Next.js (v14) framework, utilizing Tailwind CSS for a modern, responsive, and maintainable user interface.

This layer's primary responsibility is multi-faceted: it manages all client-side application state, performs initial data validation, and, crucially, acts as the secure communication proxy for the underlying Go API. The architecture is deliberately designed to offload heavy business logic to the middleware and blockchain, allowing the frontend to remain lightweight and fast.

The client application's structure is governed by two key configuration files:

- app/page.tsx (Main Application Logic)**:** This is the core component that houses the entire user dashboard and front-end logic. Key functionalities include the implementation of several critical handlers like User State Management which is the component manages the authentication state (isLoggedIn, authToken) and stores the user's role and identity claims extracted after successful login. The second one is Data Retrieval which makes the loadData and utility functions orchestrate calls to the backend, specifically fetching the dynamic list of members (from PostgreSQL) and the crucial list of pending approval actions (via the API's specialized LevelDB-compatible query endpoint). The third one is Transaction Initiation contains the handleFileUpload function demonstrates the security protocol. It receives the raw file, packages it as multipart form data (the necessary format for file transmission), and submits it to the API. It is critical to note that the hashing process occurs server-side, reinforcing security. The fourth one is Consensus Action which contains the interface provides the explicit Approve buttons, triggering the final transaction signing process. This confirms that the user explicitly chose to approve the transaction, enforcing non-repudiation.
- next.config.mjs (Proxy Configuration): This configuration file is absolutely critical for demonstrating the application's viability in a multi-user environment. It utilizes Next.js's

rewrites functionality to implement a reverse proxy. This configuration instructs the server to transparently redirect all client-side API requests originating from the external web (e.g., a team member's mobile phone using ngrok) to the internal Go server running at http://127.0.0.1:8080. This successfully bypasses cross-origin resource sharing (CORS) security issues and allows the application to be tested externally without compromising the security of the local development machine. This architectural decision confirms the operational feasibility of the system in a simulated distributed setting.

The entire front end relies heavily on the JSON Web Token (JWT) passed to it by the backend upon successful authentication. This token is securely attached to every subsequent API request, ensuring that all document proposals and approval actions are cryptographically authorized, and therefore, non-repudiable across the application.

A.2 Back End

1. Architectural Role, Security, and Gateway Management:

The Backend component of VaultChain functions as the system's Logic Layer and the singular, secure Gateway to the entire data infrastructure. This component is developed using Go (Golang) with the Gin RESTful framework, selected for its high performance, robust concurrency, and stability, which are critical traits for a blockchain middleware orchestrating network transactions. The backend maintains a strict division of data responsibility, managing all mutable application state through the fast PostgreSQL database while routing high-security transactions exclusively to the Fabric network. This hybrid architecture prevents the slow, resource-intensive nature of DLT consensus from impacting user login and application responsiveness.

The backend’s foundational responsibility is establishing and maintaining the Identity Chain of Trust. This is managed through two key modules: the Auth Middleware and the Fabric Identity Loader. The middleware intercepts all protected API routes, validating the incoming JSON Web Token (JWT) to authenticate the user. Once authenticated, the middleware extracts the user's specific claims (userEmail, userRole) and attaches them to the request context. This identity context is then used by the handlers to: 1) determine authorization (e.g.,

only Admin can view certain data), and 2) inject the verifiable, non-repudiable identity into the Chaincode transaction signature. Furthermore, the Fabric Identity Loader is responsible for loading the administrator's X.509 certificate and private key from disk to enroll the "appUser" identity into the Fabric SDK, thus enabling the Go API to securely sign and submit transactions to the peer network.

2. Core Handler Logic and Data Orchestration:

The operational value of the backend is executed within the API handlers, which perform specific data and transaction orchestration duties:

- **Document Hashing and Proposal (ProposeDocumentHandler):** This handler manages the entry of a document into the immutable workflow. It receives the document file via multipart form data (a necessary protocol fix for file uploads). The handler then calculates the document's unique SHA-256 cryptographic hash on the server side. This hash, along with the document's metadata (Name, Workspace) and the authenticated proposer's email (from the JWT), is packaged into the transaction proposal submitted to the Chaincode. The raw file is explicitly discarded after hashing, adhering to the optimized hybrid data model.
- **Consensus and Approval (ApproveDocumentHandler):** This handler routes user votes. It retrieves the unique document ID and the approver's identity from the JWT. It calls the Chaincode's ApproveDocument function, submitting a digitally signed transaction. The handler ensures that the final response to the user reflects the Chaincode's immutable determination (e.g., "Document Approved" or "User not authorized").
- **Data Retrieval and Query Optimization (GetPendingDocumentsHandler):** To feed the frontend dashboard and notification system, this handler calls the chaincode's specialized LevelDB-compatible query function (QueryPendingForUser). This function avoids unstable rich query selectors (which often fail on LevelDB) by utilizing the efficient GetStateByRange method to iterate over and filter assets. This guarantees that the system reliably displays pending approvals to the correct user without experiencing query failures, thus solving a known production stability challenge.

A.3 Source Code

This section provides the complete code for the VaultChain smart contract. These files were deployed using the Vendor method to ensure reliability in the LevelDB environment.

```
package main

import (
	"encoding/json"
	"fmt"
	"time"
	"github.com/hyperledger/fabric-contract-api-go/contractapi"
)

// SmartContract provides functions for managing the document vault.
type SmartContract struct {
	contractapi.Contract
}

// DocumentAsset describes the structure of a document stored on the ledger.
type DocumentAsset struct {
	ID              string   `json:"ID"`
	Name            string   `json:"Name"`
	Hash            string   `json:"Hash"`
	Workspace       string   `json:"Workspace"`
	Owner           string   `json:"Owner"`
	UploadTimestamp string   `json:"UploadTimestamp"`
	Status          string   `json:"Status"`
	RequiredApprovers []string `json:"RequiredApprovers"`
	ActualApprovers   []string `json:"ActualApprovers"`
	StorageID       string   `json:"StorageID"`
```

```
}

// MemberProposal represents an on-chain proposal to add a new member.

type MemberProposal struct {

	ProposalID       string   `json:"ProposalID"`

	Email            string   `json:"Email"`

	Role             string   `json:"Role"`

	Status           string   `json:"Status"`

	Approvers        []string `json:"Approvers"`

	RequiredApprovers []string `json:"RequiredApprovers"`

}

// Helper wrapper to identify asset types in the ledger

type AssetWrapper struct {

	DocType string          `json:"docType"`

	Doc     *DocumentAsset  `json:"doc,omitempty"`

	Prop    *MemberProposal `json:"prop,omitempty"`

}

// ProposeDocument creates a new document asset on the ledger.

// UPDATED: Now accepts 'requiredApproversJSON' to allow dynamic approvers from the
backend.

func (s *SmartContract) ProposeDocument(ctx contractapi.TransactionContextInterface, id
string, name string, hash string, workspace string, storageID string, proposerEmail string,
requiredApproversJSON string) error {

	exists, err := s.AssetExists(ctx, id)

	if err != nil {

		return err

	}
```

```
if exists {
	return fmt.Errorf("the document %s already exists", id)
}
// --- DYNAMIC APPROVER LOGIC ---
// Deserialize the list passed from the backend instead of hardcoding names
var requiredApprovers []string
err = json.Unmarshal([]byte(requiredApproversJSON), &requiredApprovers)
if err != nil {
	return fmt.Errorf("failed to parse required approvers: %v", err)
}
// Safety check: Ensure there is at least one approver (usually the proposer)
if len(requiredApprovers) == 0 {
	requiredApprovers = []string{proposerEmail}
}
// Auto-add proposer to actual approvers if they are in the required list
actualApprovers := []string{}
for _, required := range requiredApprovers {
	if required == proposerEmail {
		actualApprovers = append(actualApprovers, proposerEmail)
		break
	}
}
doc := DocumentAsset{
	ID:		id,
	Name:		name,
```

```
            Hash:            hash,
            Workspace:       workspace,
            Owner:           proposerEmail,
            UploadTimestamp:  time.Now().UTC().Format(time.RFC3339),
            Status:          "PendingApproval",
            RequiredApprovers: requiredApprovers, // Uses the dynamic list
            ActualApprovers:  actualApprovers,
            StorageID:       storageID,
        }
        fmt.Printf("ProposeDocument: Document ID: %s, Owner: %s, Status: %s,
Required: %v, Actual: %v\n",
            doc.ID, doc.Owner, doc.Status, doc.RequiredApprovers, doc.ActualApprovers)

        // Wrap it to distinguish from proposals
        wrapper := AssetWrapper{
            DocType: "DocumentAsset",
            Doc:     &doc,
        }
        docJSON, err := json.Marshal(wrapper)
        if err != nil {
            return err
        }

        return ctx.GetStub().PutState(id, docJSON)
}
// ApproveDocument allows a required member to approve a document.
```

```
func (s *SmartContract) ApproveDocument(ctx contractapi.TransactionContextInterface, id
string, approverEmail string) error {
	wrapperJSON, err := ctx.GetStub().GetState(id)
	if err != nil {
		return fmt.Errorf("failed to read from world state: %v", err)
	}
	if wrapperJSON == nil {
		return fmt.Errorf("the document %s does not exist", id)
	}
	var wrapper AssetWrapper
	err = json.Unmarshal(wrapperJSON, &wrapper)
	if err != nil {
		return err
	}
	// Safety check
	if wrapper.Doc == nil {
		return fmt.Errorf("asset %s is not a document", id)
	}
	doc := wrapper.Doc
	if doc.Status == "Verified" {
		return fmt.Errorf("document %s is already verified", id)
	}
	isRequired := false
	for _, required := range doc.RequiredApprovers {
		if required == approverEmail {
			isRequired = true
```

```
                        break
                }
        }
        if !isRequired {
                return fmt.Errorf("user %s is not authorized to approve this document",
approverEmail)
        }

        for _, a := range doc.ActualApprovers {
                if a == approverEmail {
                        return fmt.Errorf("user %s has already approved this document",
approverEmail)
                }
        }

        doc.ActualApprovers = append(doc.ActualApprovers, approverEmail)
        if len(doc.ActualApprovers) >= len(doc.RequiredApprovers) {
                doc.Status = "Verified"
        }
        wrapper.Doc = doc
        updatedJSON, err := json.Marshal(wrapper)
        if err != nil {
                return err
        }
        return ctx.GetStub().PutState(id, updatedJSON)
}
```

```
// ProposeNewMember creates an on-chain record for a new member proposal.
// UPDATED: Now accepts requiredApproversJSON
func (s *SmartContract) ProposeNewMember(ctx contractapi.TransactionContextInterface,
proposalID string, email string, role string, requiredApproversJSON string) error {
	// Deserialize the approvers list passed from Backend
	var requiredApprovers []string
	err := json.Unmarshal([]byte(requiredApproversJSON), &requiredApprovers)
	if err != nil {
		return fmt.Errorf("failed to parse required approvers: %v", err)
	}
	if len(requiredApprovers) == 0 {
		return fmt.Errorf("no approvers specified")
	}
	proposal := MemberProposal{
		ProposalID:       proposalID,
		Email:            email,
		Role:             role,
		Status:           "Pending",
		Approvers:        []string{},
		RequiredApprovers: requiredApprovers, // Uses the real admins now
	}

	wrapper := AssetWrapper{
		DocType: "MemberProposal",
		Prop:    &proposal,
	}
```

```
	proposalJSON, err := json.Marshal(wrapper)
	if err != nil {
		return err
	}
	return ctx.GetStub().PutState(proposalID, proposalJSON)
}
// ApproveNewMember adds an approver's vote to a member proposal.
func (s *SmartContract) ApproveNewMember(ctx contractapi.TransactionContextInterface,
proposalID string, approverEmail string) ([]byte, error) {
	// DEBUG LOG 1
	fmt.Printf("CHAINCODE DEBUG: Attempting to approve proposal %s by %s\n",
proposalID, approverEmail)
	wrapperJSON, err := ctx.GetStub().GetState(proposalID)
	if err != nil {
		return nil, fmt.Errorf("failed to read proposal: %v", err)
	}
	if wrapperJSON == nil {
		fmt.Printf("CHAINCODE ERROR: Proposal %s does not exist\n",
proposalID)
		return nil, fmt.Errorf("proposal %s does not exist", proposalID)
	}
	var wrapper AssetWrapper
	json.Unmarshal(wrapperJSON, &wrapper)
	if wrapper.Prop == nil {
		return nil, fmt.Errorf("asset %s is not a member proposal", proposalID)
	}
```

```
	proposal := wrapper.Prop
	// DEBUG LOG 2: Show who is allowed
	fmt.Printf("CHAINCODE DEBUG: Required Approvers: %v\n",
proposal.RequiredApprovers)


	if proposal.Status == "Approved" {
		return nil, fmt.Errorf("proposal %s has already been approved", proposalID)
	}
	isRequired := false
	for _, required := range proposal.RequiredApprovers {
		if required == approverEmail {
			isRequired = true
			break
		}
	}
	if !isRequired {
		// DEBUG LOG 3: Explicit failure
		fmt.Printf("CHAINCODE FAILURE: User %s is NOT in required list %v\n",
approverEmail, proposal.RequiredApprovers)
		return nil, fmt.Errorf("user %s is not authorized to approve this proposal",
approverEmail)
	}
	// Check if already approved
	for _, approver := range proposal.Approvers {
		if approver == approverEmail {
			return nil, fmt.Errorf("user %s has already approved this proposal",
approverEmail)
```

```
		}
	}

	proposal.Approvers = append(proposal.Approvers, approverEmail)

	if len(proposal.Approvers) >= len(proposal.RequiredApprovers) {
		proposal.Status = "Approved"
	}
	wrapper.Prop = proposal
	updatedJSON, _ := json.Marshal(wrapper)
	err = ctx.GetStub().PutState(proposalID, updatedJSON)
	if err != nil {
		return nil, err
	}
	fmt.Printf("CHAINCODE SUCCESS: Proposal %s approved by %s\n", proposalID,
approverEmail)
	return updatedJSON, nil
}

// QueryPendingForUser finds all assets awaiting a specific user's approval.
func (s *SmartContract) QueryPendingForUser(ctx contractapi.TransactionContextInterface,
userEmail string) (string, error) {
	resultsIterator, err := ctx.GetStub().GetStateByRange("", "")
	if err != nil {
		return "", err
	}
```

```
	defer resultsIterator.Close()


	var pendingDocs []*DocumentAsset


	for resultsIterator.HasNext() {
		queryResponse, err := resultsIterator.Next()
		if err != nil {
			return "", err
		}
		var wrapper AssetWrapper
		err = json.Unmarshal(queryResponse.Value, &wrapper)
		if err != nil {
			continue
		}
		if wrapper.DocType == "DocumentAsset" && wrapper.Doc != nil &&
wrapper.Doc.Status == "PendingApproval" {
			doc := wrapper.Doc
			isRequired := false
			for _, r := range doc.RequiredApprovers {
				if r == userEmail {
					isRequired = true
					break
				}
			}
			hasVoted := false
			for _, a := range doc.ActualApprovers {
```

```
                        if a == userEmail {
                                hasVoted = true
                                break
                        }
                }
                if isRequired && !hasVoted {
                        pendingDocs = append(pendingDocs, doc)
                }
        }
    }
    if pendingDocs == nil {
        pendingDocs = []*DocumentAsset{}
    }
    resultJSON, err := json.Marshal(pendingDocs)
    if err != nil {
        return "", fmt.Errorf("failed to marshal pending documents: %v", err)
    }
    return string(resultJSON), nil
}
// QueryAllDocuments returns ALL documents from the ledger (for history).
func (s *SmartContract) QueryAllDocuments(ctx contractapi.TransactionContextInterface)
(string, error) {
    resultsIterator, err := ctx.GetStub().GetStateByRange("", "")
    if err != nil {
        return "", err
    }
```

```
defer resultsIterator.Close()
var allDocs []*DocumentAsset
for resultsIterator.HasNext() {
	queryResponse, err := resultsIterator.Next()
	if err != nil {
		return "", err
	}
	var wrapper AssetWrapper
	err = json.Unmarshal(queryResponse.Value, &wrapper)
	if err != nil {
		continue
	}
	// Filter for DocumentAsset types only
	if wrapper.DocType == "DocumentAsset" && wrapper.Doc != nil {
		allDocs = append(allDocs, wrapper.Doc)
	}
}
if allDocs == nil {
	allDocs = []*DocumentAsset{}
}
resultJSON, err := json.Marshal(allDocs)
if err != nil {
	return "", fmt.Errorf("failed to marshal documents: %v", err)
}
return string(resultJSON), nil
```

```
}

// GetAssetHistory returns the chain of custody for an asset since creation.

func (s *SmartContract) GetAssetHistory(ctx contractapi.TransactionContextInterface, id
string) (string, error) {

	resultsIterator, err := ctx.GetStub().GetHistoryForKey(id)

	if err != nil {

		return "", err

	}

	defer resultsIterator.Close()

	var history []map[string]interface{}

	for resultsIterator.HasNext() {

		response, err := resultsIterator.Next()

		if err != nil {

			return "", err

		}

		// FIX: Unmarshal into a generic map to capture the full AssetWrapper
structure

		var asset map[string]interface{}

		if len(response.Value) > 0 {

			json.Unmarshal(response.Value, &asset)

		}

		entry := map[string]interface{}{

			"txId":      response.TxId,

			"timestamp": time.Unix(response.Timestamp.Seconds,
int64(response.Timestamp.Nanos)).Format(time.RFC3339),

			"isDelete":  response.IsDelete,
```

```
                "data":     asset, // This will now correctly contain { "prop": { "Status": "..." } }
            }
            history = append(history, entry)
        }
        resultJSON, err := json.Marshal(history)
        if err != nil {
            return "", err
        }
        return string(resultJSON), nil
}
// --- Helper Functions ---
func (s *SmartContract) ReadDocument(ctx contractapi.TransactionContextInterface, id string) (*DocumentAsset, error) {
        wrapperJSON, err := ctx.GetStub().GetState(id)
        if err != nil {
            return nil, fmt.Errorf("failed to read: %v", err)
        }
        if wrapperJSON == nil {
            return nil, fmt.Errorf("asset %s does not exist", id)
        }
        var wrapper AssetWrapper
        err = json.Unmarshal(wrapperJSON, &wrapper)
        if wrapper.Doc == nil {
            return nil, fmt.Errorf("asset %s is not a document", id)
        }
```

```
	return wrapper.Doc, nil
}
func (s *SmartContract) AssetExists(ctx contractapi.TransactionContextInterface, id string)
(bool, error) {
	assetJSON, err := ctx.GetStub().GetState(id)
	if err != nil {
		return false, fmt.Errorf("failed to read from world state: %v", err)
	}
	return assetJSON != nil, nil
}
func main() {
	chaincode, err := contractapi.NewChaincode(&SmartContract{})
	if err != nil {
		fmt.Printf("Error creating vault chaincode: %v", err)
		return
	}
	if err := chaincode.Start(); err != nil {
		fmt.Printf("Error starting vault chaincode: %v", err)
	}
}
```

A.4 Installation Procedure

The successful deployment of the VaultChain application requires a precise, synchronized execution across three separate WSL terminals, ensuring the stability of the local network environment against common dependency and configuration conflicts. This procedure begins with verifying a clean slate, followed by the sequential startup of the Hyperledger Fabric blockchain, the PostgreSQL database, and the two application servers. Initially, all running

containers are stopped (./network.sh down) to prepare Terminal 1. We explicitly start the network using the default command (./network.sh up createChannel), relying on the robust LevelDB database architecture to prevent the connection and resource failures associated with the heavier CouchDB setup. Once the channel is established, a crucial manual intervention is performed to resolve the Go SDK's failure to find the X.509 certificate: the script-generated certificate file (*-cert.pem) is renamed to the expected cert.pem via the mv command. Following this security alignment, the LevelDB-optimized chaincode, containing the multi-member consensus logic, is deployed using ./network.sh deployCC.

Simultaneously, Terminal 2 is initialized to prepare the API and data layers. After resetting the PostgreSQL container (docker compose down -v and up -d), the goose migration tool is executed to apply the SQL schema, creating the necessary users and organizations tables in the off-chain database. A final, critical step in this terminal is clearing the local identity cache (rm -rf wallet), which forces the Go API server to enroll a fresh, valid identity using the newly named cert.pem. Only after verifying "Successfully connected to Fabric Gateway" is the Go API server started with go run main.go. Finally, Terminal 3 launches the React client using npm run dev. This synchronized procedure guarantees that every component—from the stable LevelDB state database to the secure JWT middleware—is correctly linked, eliminating the "unknown authority" errors and establishing a functional environment for multi-user testing and verification.

.....*****.....